

LAPORAN PRATIUM PEMROGRAMAN ALGORITMA DAN PEMROGRAMAN
“SINGLE LINKED LIST”



disusun Oleh:

Nama : Alhamd Ridho Putra

NIM : 2511532020

Dosen Pengampu : Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya, laporan praktikum Struktur Data dengan materi "Single Linked List" ini dapat diselesaikan tepat pada waktunya.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban tertulis atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai syarat kelengkapan tugas mata kuliah Struktur Data. Tujuan utama penulisan laporan ini adalah untuk mendokumentasikan proses, hasil, serta pemahaman penulis mengenai implementasi Single Linked List, sebuah konsep fundamental dalam pengelolaan data dinamis.

Dalam proses penyusunannya, penulis menyadari bahwa laporan ini tidak lepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Wahyudi, S.T, M.T., selaku dosen pengampu mata kuliah Struktur Data yang telah memberikan bimbingan dan ilmu yang berharga selama perkuliahan dan praktikum.
2. Kakak Sherly Sukmadira, selaku asisten laboratorium yang telah membantu dan memandu jalannya praktikum.
3. Seluruh rekan-rekan mahasiswa yang telah memberikan dukungan, semangat, serta kerja sama yang baik selama proses praktikum dan penyusunan laporan ini.
4. Semua pihak yang tidak dapat disebutkan satu per satu, yang telah berkontribusi dalam kelancaran penyelesaian laporan ini.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang bersifat membangun sangat penulis harapkan sebagai bahan perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat bagi para pembaca, khususnya dalam memahami konsep dan implementasi Single Linked List dalam bidang Struktur Data.

Padang, 30 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	
DAFTAR ISI.....	
BAB I PENDAHULUAN.....	
1.1. Latar Belakang.....	
1.2. Tujuan Praktikum.....	
1.3. Manfaat Praktikum	
BAB II PEMBAHASAN.....	
2.1. Uraian Masalah	
2.2. Langkah Kerja	
2.3. Analisis Hasil	
BAB III KESIMPULAN.....	
3.1. Kesimpulan	
3.2. Saran Pengembangan	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman, efisiensi pengelolaan data merupakan aspek krusial. Struktur data linier seperti *array* sering digunakan karena kemudahannya, namun *array* memiliki keterbatasan pada ukuran yang statis dan inefisiensi saat melakukan operasi penyisipan (*insertion*) atau penghapusan (*deletion*) di tengah elemen [1]. Untuk mengatasi kendala tersebut, konsep *Linked List* hadir sebagai alternatif yang lebih dinamis.

Single Linked List adalah kumpulan simpul (*node*) yang tersusun secara berurutan, di mana setiap simpul terdiri dari dua bagian utama: data dan penunjuk (*pointer*) yang mengarah ke simpul berikutnya dalam urutan tersebut [2]. Penggunaan *Single Linked List* memungkinkan pengalokasian memori secara dinamis saat program berjalan (*runtime*), sehingga lebih fleksibel dalam menangani data yang jumlahnya berubah-ubah. Memahami implementasi *Single Linked List* sangat penting bagi mahasiswa informatika sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti *Stack*, *Queue*, hingga *Graph*.

1.2. Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar dan struktur dari *Single Linked List*.
2. Mampu mengimplementasikan operasi-operasi dasar pada *Single Linked List*, seperti *insert* (depan, tengah, belakang), *delete*, dan *traversal*.
3. Menganalisis perbedaan efisiensi antara penggunaan *array* dan *Linked List* dalam skenario pemrograman tertentu.
4. Meningkatkan kemampuan pemecahan masalah melalui logika pemrograman struktur data dinamis.

1.3. Manfaat Praktikum

Praktikum ini diharapkan dapat memberikan manfaat sebagai berikut:

- **Bagi Mahasiswa:** Meningkatkan pemahaman teknis mengenai manajemen memori dan manipulasi *pointer* dalam bahasa pemrograman (seperti Java atau C++).
- **Bagi Pengembangan Skill:** Menjadi bekal dasar dalam membangun aplikasi yang memerlukan pengelolaan data yang efisien dan dinamis.
- **Bagi Akademik:** Sebagai dokumentasi bukti pemahaman terhadap materi mata kuliah Struktur Data.

BAB II PEMBAHASAN

2.1. Uraian Masalah

Praktikum ini berfokus pada implementasi *Single Linked List* (SLL) menggunakan bahasa pemrograman Java. SLL adalah struktur data linier di mana elemen-elemennya tidak disimpan di lokasi memori yang berdekatan, melainkan dihubungkan menggunakan referensi objek atau *pointer* [1].

Berikut adalah uraian dari kode program yang diimplementasikan:

1. Struktur Node (NodeSLL_2020):

Ini adalah komponen dasar pembentuk *list*. Kelas ini memiliki dua atribut: data bertipe integer untuk menyimpan nilai, dan next yang merupakan referensi ke objek NodeSLL_2020 berikutnya. Konstruktor digunakan untuk menginisialisasi data dan mengatur next menjadi null.

package pekan5_2511532020;

```
public class NodeSLL_2020 {  
    int data;          // Untuk menyimpan angka  
    NodeSLL_2020 next; // Untuk menyimpan alamat/rujukan ke node  
                       berikutnya  
  
    // Constructor untuk membuat node baru  
    public NodeSLL_2020(int data_2020) {  
        this.data = data_2020;  
        this.next = null;  
    }  
}
```

2. Operasi Penambahan (TambahSLL_2511532020):

- insertAtFront: Menambahkan *node* baru di awal *list* dengan mengarahkan next dari *node* baru ke head yang lama.

- insertAtEnd: Menambahkan *node* di akhir dengan melakukan *traversal* hingga menemukan *node* yang next-nya bernilai null.
- insertPos: Menyisipkan *node* pada posisi tertentu. Jika posisi adalah 1, ia memanggil logika *insert front*. Jika tidak, program mencari posisi \$n-1\$ dan mengubah rujukan *pointer* agar menyertakan *node* baru di tengah urutan.

```
package pekan5_2511532020;

public class TambahSLL_2511532020 {

    public static NodeSLL_2020 insertAtFront(NodeSLL_2020 head_2020,
int value_2020) {
        NodeSLL_2020 new_node_2020 = new NodeSLL_2020(value_2020);
        new_node_2020.next = head_2020;
        return new_node_2020;
    }

    public static NodeSLL_2020 insertAtEnd(NodeSLL_2020 head_2020,
int value_2020) {
        NodeSLL_2020 newNode_2020 = new NodeSLL_2020(value_2020);
        if (head_2020 == null) {
            return newNode_2020;
        }
        NodeSLL_2020 last_2020 = head_2020;
        while (last_2020.next != null) {
            last_2020 = last_2020.next;
        }
        last_2020.next = newNode_2020;
        return head_2020;
    }

    static NodeSLL_2020 GetNode(int data_2020) {
        return new NodeSLL_2020(data_2020);
    }

    static NodeSLL_2020 insertPos(NodeSLL_2020 headNode_2020, int
position_2020, int value_2020) {
        if (position_2020 < 1) {
            System.out.println("Invalid Position");
            return headNode_2020;
        }

        // Jika tambah di posisi pertama (head)
        if (position_2020 == 1) {
            NodeSLL_2020 newNode_2020 = new
NodeSLL_2020(value_2020);
            newNode_2020.next = headNode_2020;
            return newNode_2020;
        }

        NodeSLL_2020 temp_2020 = headNode_2020;
        // Iterasi sampai posisi sebelum lokasi penyisipan
```



```

        for (int i = 1; i < position_2020 - 1 && temp_2020 != null;
i++) {
            temp_2020 = temp_2020.next;
        }

        if (temp_2020 == null) {
            System.out.println("Posisi di luar jangkauan");
        } else {
            NodeSLL_2020 newNode_2020 = GetNode(value_2020);
            newNode_2020.next = temp_2020.next;
            temp_2020.next = newNode_2020;
        }
        return headNode_2020;
    }

    public static void printList_2020(NodeSLL_2020 head) {
        NodeSLL_2020 curr_2020 = head;
        while (curr_2020 != null) {
            System.out.print(curr_2020.data + (curr_2020.next !=
null ? "-->" : ""));
            curr_2020 = curr_2020.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // Menggunakan class NodeSLL_2020 secara konsisten
        NodeSLL_2020 head_2020 = new NodeSLL_2020(2);
        head_2020.next = new NodeSLL_2020(3);
        head_2020.next.next = new NodeSLL_2020(5);
        head_2020.next.next.next = new NodeSLL_2020(6);

        System.out.print("Senarai berantai awal: ");
        printList_2020(head_2020);

        System.out.print("Tambah 1 simpul di belakang (7): ");
        int data2_2020 = 7;
        head_2020 = insertAtEnd(head_2020, data2_2020);
        printList_2020(head_2020);

        System.out.print("Tambah 1 simpul ke posisi 4 (data 4): ");
        int data3_2020 = 4;
        int pos_2020 = 4;
        head_2020 = insertPos(head_2020, pos_2020, data3_2020);
        printList_2020(head_2020);
    }
}

```

3. Operasi Penghapusan (HapusSLL_2511532020):

- deleteHead_2020: Menghapus elemen pertama dengan memindahkan *pointer* head ke simpul berikutnya.

- `removeLastNode_2020`: Menghapus elemen terakhir dengan mencari simpul kedua dari belakang (*second last*) dan mengubah rujukan `next`-nya menjadi `null`.
- `deleteNode_2020`: Menghapus simpul pada posisi spesifik dengan cara menghubungkan simpul sebelumnya langsung ke simpul setelah simpul yang dihapus.

```
package pekan5_2511532020;

public class HapusSLL_2511532020 {

    public static NodeSLL_2020 deleteHead_2020(NodeSLL_2020
head_2020) {
        if (head_2020 == null) return null;
        return head_2020.next;
    }

    public static NodeSLL_2020 removeLastNode_2020(NodeSLL_2020
head_2020) {
        // Cek jika list kosong atau hanya punya 1 simpul
        if (head_2020 == null || head_2020.next == null) {
            return null;
        }

        NodeSLL_2020 secondLast_2020 = head_2020;
        while (secondLast_2020.next.next != null) {
            secondLast_2020 = secondLast_2020.next;
        }
        secondLast_2020.next = null;
        return head_2020;
    }

    public static NodeSLL_2020 deleteNode_2020(NodeSLL_2020
head_2020, int position_2020) {
        if (head_2020 == null) return null;

        // Jika hapus posisi pertama
        if (position_2020 == 1) {
            return head_2020.next;
        }

        NodeSLL_2020 temp_2020 = head_2020;
        NodeSLL_2020 prev_2020 = null;

        // Cari node pada posisi yang diinginkan
        for (int i = 1; temp_2020 != null && i < position_2020; i++)
        {
            prev_2020 = temp_2020;
            temp_2020 = temp_2020.next;
        }

        // Jika posisi ditemukan
        if (temp_2020 != null && prev_2020 != null) {
```

```

        prev_2020.next = temp_2020.next;
    } else {
        System.out.println("Posisi " + position_2020 + " tidak
ditemukan.");
    }

    return head_2020;
}

public static void printList_2020(NodeSLL_2020 head_2020) {
    if (head_2020 == null) {
        System.out.println("List kosong.");
        return;
    }
    NodeSLL_2020 curr_2020 = head_2020;
    while (curr_2020 != null) {
        System.out.print(curr_2020.data + (curr_2020.next !=
null ? "-->" : ""));
        curr_2020 = curr_2020.next;
    }
    System.out.println();
}

public static void main(String[] args) {
    NodeSLL_2020 head_2020 = new NodeSLL_2020(1);
    head_2020.next = new NodeSLL_2020(2);
    head_2020.next.next = new NodeSLL_2020(3);
    head_2020.next.next.next = new NodeSLL_2020(4);
    head_2020.next.next.next.next = new NodeSLL_2020(5);
    head_2020.next.next.next.next.next = new NodeSLL_2020(6);

    System.out.print("List awal: ");
    printList_2020(head_2020);

    head_2020 = deleteHead_2020(head_2020);
    System.out.print("Setelah head dihapus: ");
    printList_2020(head_2020);

    head_2020 = removeLastNode_2020(head_2020);
    System.out.print("Setelah simpul terakhir dihapus: ");
    printList_2020(head_2020);

    int pos_2020 = 2;
    head_2020 = deleteNode_2020(head_2020, pos_2020);
    System.out.print("Setelah posisi " + pos_2020 + " dihapus:
");
    printList_2020(head_2020);
}
}

```

4. Pencarian dan Penelusuran (PencarianSLL_2511532020):

- searchKey_2020: Melakukan pencarian linier. Program memeriksa setiap simpul satu per satu hingga data yang dicari ditemukan

(mengembalikan true) atau hingga mencapai akhir *list* (mengembalikan false) [2].

- o traversal_2020: Mengunjungi setiap simpul dan mencetak datanya ke layar.

```
package pekan5_2511532020;

public class PencarianSLL_2511532020 {
    static boolean searchKey_2020(NodeSLL_2020 head_2020, int
key_2020) {
        NodeSLL_2020 curr_2020 = head_2020;
        while (curr_2020 != null) {
            if (curr_2020.data == key_2020)
                return true;
            curr_2020 = curr_2020.next;
        }
        return false;
    }

    public static void traversal_2020(NodeSLL_2020 head_2020) {
        NodeSLL_2020 curr_2020 = head_2020;
        while (curr_2020 != null) {
            System.out.print(" " + curr_2020.data);
            curr_2020 = curr_2020.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2020 head_2020 = new NodeSLL_2020(14);
        head_2020.next = new NodeSLL_2020(21);
        head_2020.next.next = new NodeSLL_2020(13);
        head_2020.next.next.next = new NodeSLL_2020(30);
        head_2020.next.next.next.next = new NodeSLL_2020(10);
        System.out.print("Penelusuran SLL : ");
        traversal_2020(head_2020);
        int key_2020 = 30;
        System.out.print("cari data " + key_2020 + " = ");
        if (searchKey_2020(head_2020, key_2020))
            System.out.println("Ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

2.2. Langkah Kerja

Langkah-langkah yang dilakukan dalam praktikum ini adalah sebagai berikut:

1. Mendefinisikan kelas simpul (NodeSLL_2020) sebagai representasi data dan *pointer*.

2. Membuat kelas utama untuk operasi penambahan, di mana *list* awal dibentuk secara manual kemudian diuji dengan metode penyisipan di akhir dan di posisi tertentu.
3. Mengimplementasikan kelas pencarian untuk memastikan data yang telah dimasukkan dapat ditemukan kembali melalui proses *looping* (perulangan *while*).
4. Mengimplementasikan kelas penghapusan untuk menguji ketahanan struktur data saat elemen di awal, tengah, atau akhir dibuang.
5. Melakukan kompilasi dan menjalankan setiap kelas melalui metode main untuk melihat *output* pada konsol.

2.3. Analisis Hasil

Berdasarkan hasil eksekusi program, dapat dianalisis beberapa poin penting:

- **Efisiensi Pencarian:** Metode `searchKey_2020` memiliki kompleksitas waktu $O(n)$, karena dalam kondisi terburuk (*worst case*), program harus memeriksa seluruh simpul jika data berada di akhir atau tidak ada dalam *list*.
- **Fleksibilitas Penambahan:** Berbeda dengan *array* yang memerlukan pergeseran elemen saat menyisipkan data di tengah, *Single Linked List* hanya perlu mengubah rujukan dua *pointer*, yang secara teoritis lebih efisien jika posisi simpul sudah diketahui [1].
- **Manajemen Memori:** Program menunjukkan bahwa penghapusan simpul di Java cukup dilakukan dengan memutus rujukan *pointer*. Simpul yang tidak lagi memiliki rujukan akan secara otomatis dibersihkan oleh *Garbage Collector*.
- **Akurasi Posisi:** Pada metode `insertPos` dan `deleteNode`, penggunaan indeks dimulai dari 1 (sesuai logika manusia). Diperlukan pengecekan kondisi null (seperti `if (temp_2020 == null)`) untuk mencegah *NullPointerException* jika pengguna memasukkan posisi di luar jangkauan *list*.

BAB III

KESIMPULAN

3.1. Kesimpulan

Berdasarkan hasil praktikum mengenai *Single Linked List* (SLL) yang telah dilaksanakan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. *Single Linked List* merupakan struktur data dinamis yang sangat efisien dalam penggunaan memori karena alokasi dilakukan sesuai kebutuhan saat program berjalan (*runtime*), berbeda dengan *array* yang bersifat statis.
2. Operasi pencarian (*searching*) pada SLL memiliki kompleksitas waktu linear $O(n)$ karena harus menelusuri simpul satu per satu dari head hingga data ditemukan [1].
3. Implementasi operasi penambahan dan penghapusan simpul pada posisi tertentu memerlukan ketelitian dalam manipulasi *pointer* (atau rujukan objek di Java) untuk mencegah hilangnya akses ke simpul-simpul berikutnya (*memory leak* atau *NullPointerException*).
4. Penggunaan akhiran NIM pada variabel (seperti `_2020`) membantu dalam identifikasi kepemilikan kode dan memastikan alur logika program tetap konsisten dalam lingkungan akademik [3].

3.2. Saran Pengembangan

Untuk pengembangan lebih lanjut atau praktikum berikutnya, disarankan untuk:

1. Mengembangkan struktur data ini menjadi *Doubly Linked List* agar proses penelusuran (*traversal*) dapat dilakukan secara dua arah (maju dan mundur), yang akan meningkatkan efisiensi pada operasi tertentu [5].
2. Menambahkan fungsi pengurutan (*sorting*) seperti *Merge Sort* yang secara spesifik bekerja lebih efisien pada struktur *Linked List* dibandingkan *array* [4].
3. Menerapkan penanganan eksepsi (*exception handling*) yang lebih kuat untuk memvalidasi masukan posisi agar program tidak berhenti tiba-tiba saat pengguna memasukkan indeks yang tidak valid.

TUGAS PRAKTIKUM

1. KODE

```
package pekan5_2511532020;

/**
 * Kelas ADT Pasien - Mewakili satu node dalam Single Linked List
 * Setiap node menyimpan data pasien dan referensi ke node berikutnya
 */
public class Pasien_2511532020 {
    // Atribut dengan akhiran 4 digit NIM
    private String namaPasien_2020;
    private String penyakit_2020;
    private int nomorAntrian_2020;

    // Pointer ke node berikutnya (wajib akhiran NIM)
    Pasien_2511532020 next_2020;

    /**
     * Constructor: Menginisialisasi semua atribut node pasien
     * @param nama Nama pasien
     * @param penyakit Keluhan/penyakit pasien
     * @param nomor Nomor antrian pasien
     */
    public Pasien_2511532020(String nama, String penyakit, int nomor) {
        this.namaPasien_2020 = nama;
        this.penakit_2020 = penyakit;
        this.nomorAntrian_2020 = nomor;
        this.next_2020 = null; // Awalnya tidak menunjuk ke node manapun
    }

    // ===== SELEKTOR (GETTER) =====
    public String getNama_2020() {
        return namaPasien_2020;
    }

    public String getPenyakit_2020() {
        return penyakit_2020;
    }

    public int getNomorAntrian_2020() {
        return nomorAntrian_2020;
    }
}
```

2. DRIVER

```
package pekan5_2511532020;

import java.util.Scanner;

/**
 * Kelas Driver - Mengelola simulasi antrian rumah sakit
 */
```



```

* menggunakan Single Linked List dengan metode FIFO
*/
public class RumahSakit_2511532020 {
    // Head linked list (node pertama)
    private Pasien_2511532020 head_2020;
    // Counter untuk auto-increment nomor antrian
    private int counterAntrian_2020;
    // Scanner untuk input
    private Scanner scanner_2020;

    /**
     * Constructor: Inisialisasi list kosong dan counter mulai dari 0
     */
    public RumahSakit_2511532020() {
        head_2020 = null;
        counterAntrian_2020 = 0;
        scanner_2020 = new Scanner(System.in);
    }

    /**
     * 1. DAFTARKAN PASIEN (Insert at Tail)
     * Menambahkan node baru di akhir linked list.
     * Jika list kosong, node baru menjadi head.
     * Nomor antrian bertambah otomatis (auto-increment).
     */
    public void daftarkanPasien_2020() {
        System.out.print("Masukkan Nama Pasien : ");
        String nama = scanner_2020.nextLine();
        System.out.print("Masukkan Keluhan : ");
        String keluhan = scanner_2020.nextLine();

        // Auto-increment nomor antrian
        counterAntrian_2020++;

        // Membuat node pasien baru
        Pasien_2511532020 pasienBaru = new Pasien_2511532020(nama,
keluhan, counterAntrian_2020);

        // KONDISI 1: List kosong, node baru menjadi head
        if (head_2020 == null) {
            head_2020 = pasienBaru;
        }
        // KONDISI 2: List tidak kosong, traverse ke node terakhir
        else {
            Pasien_2511532020 current = head_2020;
            // Traverse sampai node terakhir (next == null)
            while (current.next_2020 != null) {
                current = current.next_2020;
            }
            // Sambungkan node terakhir ke node baru (Insert at Tail)
            current.next_2020 = pasienBaru;
        }

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
" + counterAntrian_2020);
        System.out.println();
    }
}

```

```

/**
 * 2. PANGGIL PASIEN (Delete Head - FIFO)
 * Menghapus node terdepan dan menampilkan data pasien yang
dipanggil.
 * Head digeser ke node berikutnya.
 */
public void panggilPasien_2020() {
    // Penanganan list kosong (tidak boleh error)
    if (head_2020 == null) {
        System.out.println("== Antrian kosong! Tidak ada pasien yang
dapat dipanggil. ==");
        System.out.println();
        return;
    }

    // Simpan data pasien terdepan yang akan dihapus
    Pasien_2511532020 pasienDipanggil = head_2020;

    // Geser head ke node berikutnya (Delete Head)
    head_2020 = head_2020.next_2020;

    // Tampilkan informasi pasien yang dipanggil
    System.out.println("== MEMANGGIL PASIEN ==");
    System.out.println("Nomor Antrian : " +
pasienDipanggil.getNomorAntrian_2020());
    System.out.println("Nama Pasien : " +
pasienDipanggil.getNama_2020());
    System.out.println("Keluhan : " +
pasienDipanggil.getPenyakit_2020());
    System.out.println("Silakan menuju ruang pemeriksaan!");
    System.out.println();

    // NOTE: Node yang dihapus akan otomatis dibersihkan oleh
Garbage Collector
    // karena sudah tidak ada referensi yang menunjuk ke node
tersebut
}

/**
 * 3. TAMPILKAN ANTRIAN (Display / Traverse)
 * Menelusuri linked list dari head hingga null
 * dan menampilkan semua data pasien beserta posisinya.
 */
public void tampilkanAntrian_2020() {
    // Penanganan list kosong
    if (head_2020 == null) {
        System.out.println("== Antrian kosong! Tidak ada pasien
dalam antrian. ==");
        System.out.println();
        return;
    }

    System.out.println("===== DAFTAR ANTRIAN PASIEN
=====");
    System.out.printf("%-5s %-20s %-25s\n", "No.", "Nama Pasien",
"Keluhan");

```

```

        System.out.println("-----");
    -");

    // Traverse dari head hingga akhir list (null)
    Pasien_2511532020 current = head_2020;
    int posisi = 1;
    while (current != null) {
        System.out.printf("%-5d %-20s %-25s\n",
            posisi,
            current.getNama_2020(),
            current.getPenyakit_2020());
        current = current.next_2020; // Pindah ke node berikutnya
        posisi++;
    }
    System.out.println("-----");
-");
    System.out.println("Total pasien dalam antrian: " + (posisi -
1));
    System.out.println();
}

/**
 * 4. CARI PASIEN (Search - Case Insensitive)
 * Pencarian berdasarkan nama pasien secara linear,
 * tidak membedakan huruf besar/kecil.
 * Menampilkan pesan jika tidak ditemukan.
 */
public void cariPasien_2020() {
    // Penanganan list kosong
    if (head_2020 == null) {
        System.out.println("== Antrian kosong! Tidak ada pasien yang
dapat dicari. ==");
        System.out.println();
        return;
    }

    System.out.print("Masukkan nama pasien yang dicari: ");
    String namaCari = scanner_2020.nextLine();

    // Traverse untuk mencari pasien (Linear Search)
    Pasien_2511532020 current = head_2020;
    boolean ditemukan = false;
    int posisi = 1;

    while (current != null) {
        // Perbandingan case-insensitive (tidak membedakan huruf
besar/kecil)
        if (current.getNama_2020().equalsIgnoreCase(namaCari)) {
            System.out.println("== PASIEN DITEMUKAN ==");
            System.out.println("Posisi Antrian ke-" + posisi);
            System.out.println("Nama Pasien : " +
current.getNama_2020());
            System.out.println("Keluhan : " +
current.getPenyakit_2020());
            System.out.println("Nomor Antrian : " +
current.getNomorAntrian_2020());
            ditemukan = true;

```

```

        break; // Hentikan pencarian jika sudah ditemukan
    }
    current = current.next_2020;
    posisi++;
}

if (!ditemukan) {
    System.out.println("Pasien dengan nama \" + namaCari + "\"
tidak ditemukan dalam antrian.");
}
System.out.println();
}

/**
 * 5. CEK STATUS ANTRIAN
 * Menampilkan jumlah total pasien dan informasi pasien terdepan.
 * Menampilkan pesan khusus jika list kosong.
 */
public void cekStatusAntrian_2020() {
    // Penanganan list kosong
    if (head_2020 == null) {
        System.out.println("== Status Antrian ==");
        System.out.println("Antrian kosong! Tidak ada pasien yang
menunggu.");
        System.out.println();
        return;
    }

    // Hitung total pasien dengan traverse seluruh list
    int total = 0;
    Pasien_2511532020 current = head_2020;
    while (current != null) {
        total++;
        current = current.next_2020;
    }

    System.out.println("== Status Antrian ==");
    System.out.println("Jumlah pasien dalam antrian : " + total);
    System.out.println("--- Pasien Terdepan ---");
    System.out.println("Nama Pasien : " +
head_2020.getNama_2020());
    System.out.println("Keluhan : " +
head_2020.getPenyakit_2020());
    System.out.println("Nomor Antrian : " +
head_2020.getNomorAntrian_2020());
    System.out.println();
}

/**
 * Menampilkan menu utama program
 */
public void tampilkanMenu_2020() {
    System.out.println("== Antrian Rumah Sakit NIM: 2511532020 ==");
    System.out.println("1. Daftarkan Pasien (Insert)");
    System.out.println("2. Panggil Pasien (Delete Head)");
    System.out.println("3. Tampilkan Antrian (Display)");
    System.out.println("4. Cari Pasien (Search)");
}

```

```

        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
    }

    /**
     * Method untuk menjalankan program utama (loop menu)
     */
    public void jalankan_2020() {
        int pilihan;
        do {
            tampilkanMenu_2020();
            // Validasi input agar hanya menerima angka
            while (!scanner_2020.hasNextInt()) {
                System.out.println("Masukkan pilihan berupa angka!");
                scanner_2020.next(); // Buang input yang salah
                System.out.print("Pilihan: ");
            }
            pilihan = scanner_2020.nextInt();
            scanner_2020.nextLine(); // Membersihkan buffer newline
            System.out.println();

            // Percabangan untuk menjalankan operasi sesuai pilihan
            switch (pilihan) {
                case 1:
                    daftarkanPasien_2020();
                    break;
                case 2:
                    panggilPasien_2020();
                    break;
                case 3:
                    tampilkanAntrian_2020();
                    break;
                case 4:
                    cariPasien_2020();
                    break;
                case 5:
                    cekStatusAntrian_2020();
                    break;
                case 6:
                    System.out.println("Terima kasih! Program
selesai.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid! Silakan
coba lagi.");
                    System.out.println();
            }
        } while (pilihan != 6);

        scanner_2020.close();
    }

    /**
     * Method main - Entry point program
     */
    public static void main(String[] args) {

```

```
RumahSakit_2511532020 rs = new RumahSakit_2511532020();  
rs.jalankan_2020();  
}  
}
```

DAFTAR PUSTAKA

- [1] R. Stephens, *Beginning Data Structures and Algorithms*. Indianapolis, IN: Wiley Publishing, 2011.
- [2] N. Karumanchi, *Data Structures and Algorithms Made Easy: Data Structures and Algorithmic Puzzles*. Hyderabad: CareerMonk Publications, 2016.
- [3] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA: Pearson Education, 2012.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.
- [5] S. Sahni, *Data Structures, Algorithms, and Applications in Java*, 2nd ed. Hyderabad: Universities Press, 2005.